

Project: Classification Model Implementation

Submitted By: Shikha Raj

Report Contents

1. Problem Description:1
2. Data Selection and Understanding
3. Exploratory Data Analysis (EDA)
4. Data Cleaning
5. Model Implementation and Performance Assessment
6. Logistic Regression Implementation(Baseline Model)
7. Random Forest Implementation
8. XGBOOST Implementation
9. Binary Classification Approach(Class1: Functional, Class2: Non function + Need Repair)
10. Executive Summary

Problem Description:

To ensure that clean, potable water is available to communities across Tanzania, a clear understanding of which waterpoints require maintenance, need replacement, or are in good condition can significantly improve operational efficiency.

Using data from the Tanzanian Ministry of Water, build a model to predict one of three classes (functional, needs repair, or non-functional) for each waterpoint. This prediction is based on variables such as pump type, geographic water basin, installation date, water quality and quantity, water source, total static head (available water pressure), and local population.

Data Selection and Understanding:

I am using a dataset from Taarifa waterpoints dashboard, which aggregates data from the Tanzania Ministry of Water.

Taarifa Home Page: <https://taarifa.org>

Taarifa GitHub Page (for data): <https://github.com/taarifa>

Dataset Size: The dataset contains approximately 59,400 records and includes 40 variables, including both categorical and continuous features.

Target Variable Classes: The target variable comprises three total classes:

1. **functional** - the waterpoint is operational and there are no repairs needed
2. **functional needs repair** - the waterpoint is operational, but needs repairs
3. **non-functional** - the waterpoint is not operational.

The dataset includes **40 variables** (including the unique ID). These features can be broadly categorized into **five key information categories** influencing waterpoint functionality. Categories: Waterpoint quality, location and demographics, management, age, and source.

Variable Category	Variables
waterpoint quality	- amount_tsh - Total static head (amount water available to waterpoint) - gps_height - Altitude of the well - extraction_type - The kind of extraction the waterpoint uses - extraction_type_group - The kind of extraction the waterpoint uses - extraction_type_class - The kind of extraction the waterpoint uses - water_quality - The quality of the water - quality_group - The quality of the water - quantity - The quantity of water - quantity_group - The quantity of water
waterpoint location and Demographics	- longitude - GPS coordinate - latitude - GPS coordinate - wpt_name - Name of the waterpoint if there is one - num_private - - basin - Geographic water basin - subvillage - Geographic location - region - Geographic location - region_code - Geographic location (coded) - district_code - Geographic location (coded) - lga - Geographic location - ward - Geographic location
Management who handles waterpoint	- funder - Who funded the well - installer - Organization that installed the well - public_meeting - True/False - recorded_by - Group entering this row of data - scheme_management - Who operates the waterpoint - scheme_name - Who operates the waterpoint - permit - If the waterpoint is permitted - management - How the waterpoint is managed - management_group - How the waterpoint is managed - payment - What the water costs - payment_type - What the water costs
Age of waterpoint	- date_recorded - The date the row was entered - construction_year - Year the waterpoint was constructed
Source of waterpoint	- source - The source of the water - source_type - The source of the water - source_class - The source of the water - waterpoint_type - The kind of waterpoint - waterpoint_type_group - The kind of waterpoint

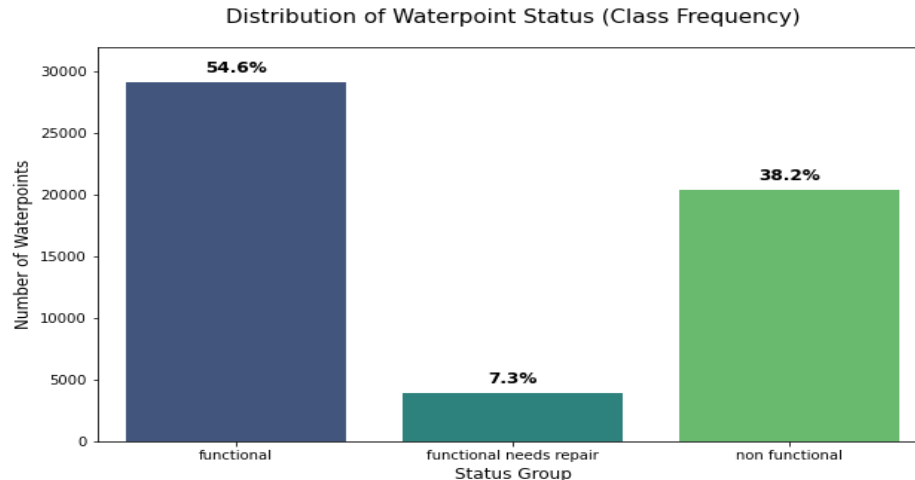
Note: I am using the same dataset utilized for the 'Linear Regression for Classification with One-Hot Encoding' project.

Key Takeaways from Data Understanding

1. The dataset consists of **40 features** (including the unique ID) and a **tri-class target variable**: *functional*, *functional needs repair*, and *non-functional*.
2. The variables can be synthesized into **five core information categories**: Water Quality, Location/Demographics, Management, Age, and Source.
3. Preliminary analysis reveals significant **feature redundancy**, where multiple variables provide overlapping information. These are removed to reduce model complexity, improve efficiency and performance.

Exploratory Data Analysis (EDA)

1. Class Distribution

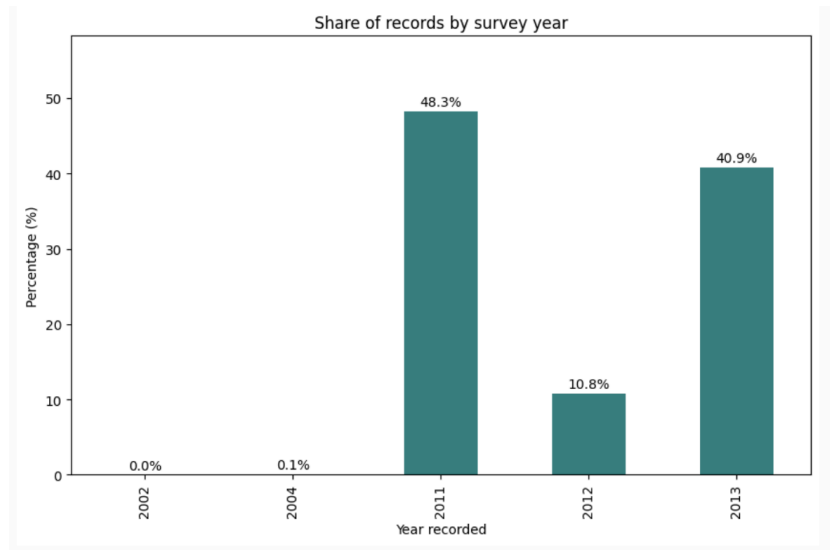


Insight:

Class Imbalance: There is a significant **class imbalance**; the 'functional needs repair' category represents only **7.3%** of the total observations. A model may struggle to identify this class and might simply default to predicting "functional" or "non-functional" to achieve a high overall accuracy. To address this issue, I will use **class weights** in the loss function to ensure the model attends to minority cases.

Cost of Misclassification: The cost of misclassification is asymmetrical for different classes. The primary **objective is to minimize 'False Negatives' for non-functional** waterpoints, as failing to identify a broken source directly denies communities access to clean water. Furthermore, misclassifying 'functional needs repair' points as functional will prevent early intervention and will lead to waterpoint failure and increase the government's long-term maintenance costs. Hence, **this project prioritizes low False Negative for the 'non-functional' and 'functional needs repair' classes** to ensure that they are flagged for immediate attention.

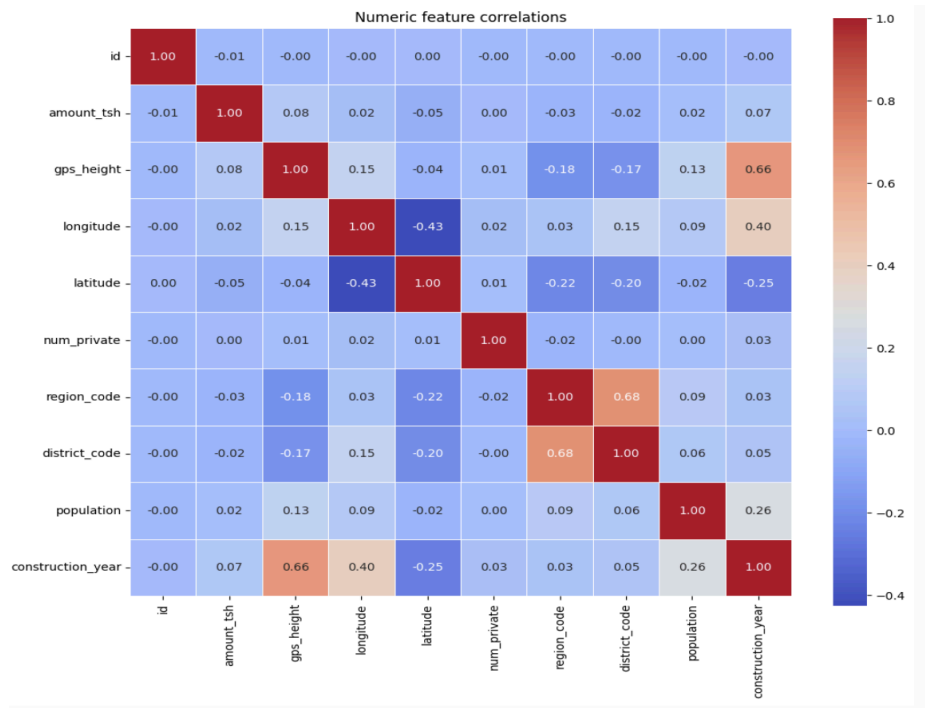
2. Share of record by survey Year



Insight:

The vast majority of records were collected between 2011 and 2013. Observations from 2002 and 2004 account for only **0.1%** of the dataset. Due to their age and frequency, these records were removed to ensure the model focuses on more recent, relevant data.

3. Share of record by survey Year



Insight:

The correlation matrix reveals that most numeric features are independent, which is ideal for a stable Logistic Regression model.

The region_code and district_code features are highly correlated because they provide overlapping geographic information. To avoid multicollinearity and improve model stability while retaining essential geographic context, I removed region_code from the data.

There is a strong relationship between GPS Height and Construction Year (0.66), suggesting a geographic trend in infrastructure development over time. Population and Amount_tsh show near-zero correlation with other variables, indicating they provide unique, non-redundant signals. Overall, there is no extreme multicollinearity that confirms that the selected features are distinct and well-suited for model implementation.

4. Other Numerical Analysis Insights

Population, longitude, district_code, and construction_year have many zero values, which constitutes invalid data for these field types. This suggests that 0 was frequently used as a placeholder when actual data was unavailable. Recognizing these placeholders is essential for maintaining the integrity of our predictive analysis and ensuring that the model does not misinterpret "missingness" as a literal numeric signal.

The amount_tsh and population columns exhibit severe skewness, with a small number of records having exceptionally high values that could disproportionately influence linear model training (such as Logistic Regression). In contrast, tree-based models (Random Forest and XGBoost) are naturally robust to such skewness and can handle these extreme values effectively without transformation, as they rely on hierarchical splitting rather than coordinate-based weights.

5. Missing values in Data

	Total Missing	Percentage (%)
population	21369	35.993532
construction_year	20695	34.858259
public_meeting	3328	5.605619
permit	3055	5.145783
longitude	1812	3.052098
district_code	23	0.038741

The dataset contains significant missing information across several key features, most notably in **population (35.99%)** and **construction_year (34.85%)**. Rather than removing these records, which would result in losing over one-third of the total observations, these fields were imputed using a logical strategy.

Note: The detailed imputation process for every data field is elaborated on and described comprehensively in the Data Cleaning section of this report.

Data Cleaning

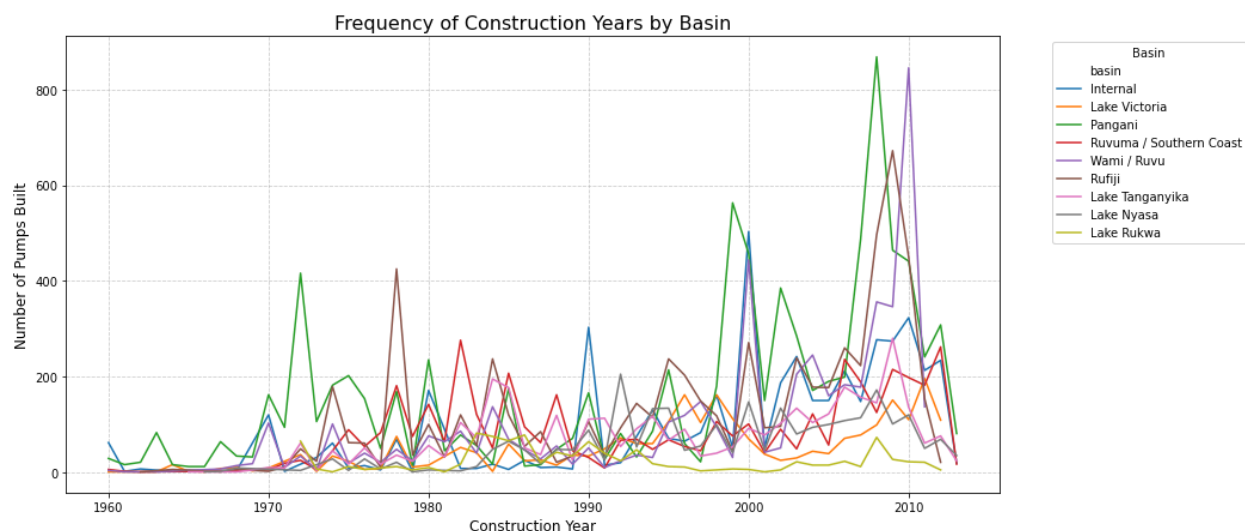
Population missing value imputation

To address missingness in population data, a Hierarchical Group-based Imputation strategy was implemented. Rather than applying a global median, missing values were first imputed using the median of the District. If District data was missing, values were imputed with the median of the region. If the region data was also absent, the median of the Basin was utilized. This multilevel approach ensures that the imputed data maintains geographic and demographic integrity.

District → Region → Basin uses nearer geography first

Construction_year missing value imputation

The graph below shows the Construction Year vs. the number of pumps built in that year across different Water Basins.



Each water basin exhibits a distinct peak in construction activity, likely representing a period of intensive infrastructure development for that region. For instance, the Pangani Basin saw its highest volume of construction in 2008. Consequently, missing values in the construction_year column can be imputed using the mode (the most frequent year) of the respective geographic basin.

Longitude missing value imputation

To address missing values in the longitude field, a hierarchical geographic imputation was applied. First, missing coordinates were filled using the median longitude of the same region and district. For any remaining missing values, a fallback logic was used to impute data based on the overall regional median. This tiered approach ensures that the geographic location of each waterpoint remains as localized as possible.

public_meeting and permit value imputation

Missing values were filled using a hierarchical logic: first by district_code mode, then by region mode, then by water basin mode of the dataset. This ensures that the imputed value reflects local administrative realities, as permit and meeting frequencies often vary by specific district or regional management.

District_code value imputation

This was imputed using a similar hierarchy starting at the region level, followed by the basin and global levels.

Other Data Cleaning

- We limited the dataset to records from 2011 to 2013 because only 0.1% of the records predated 2004. These older records appeared to be outliers and were removed to ensure the model focuses on recent, relevant data.
- Invalid Longitude = 0 not located in Tanzania were set to NaN.
- population cannot be 0; these values were set to NaN.
- Construction Year cannot be 0; since 0 indicates the year is unknown, these were set to NaN.
- High-cardinality variables (such as ward, region_code, and district_code) were removed to simplify the model and reduce noise.
- Variables providing similar information were removed, retaining only one of each to address multicollinearity issues.
- Constant variables, for which all values are the same (such as recorded_by), were removed.
- Variables where most values are NaN (such as scheme_name) were removed.

Feature Extraction and Transformation

Derived Variable: A new variable, age_of_waterpoint, was created (date_recorded - construction_year). This derived variable is more informative for determining the health of a waterpoint than using date_recorded and construction_year as separate features.

Standardization: Continuous variables were standardized to bring them to a common scale for the linear regression model. This avoids bias caused by large numerical values in any specific continuous variable.

Model Implementation and Performance Assessment

Train & Test: 80-20 split

The dataset was divided into a training set (80%) and an independent holdout test set (20%). A **stratified sampling technique was applied** to this split to ensure that the distribution of the target classes remains consistent across both subsets. By maintaining these proportions, we

prevent the model from becoming biased toward the majority class and ensure it learns the characteristics of the critical "functional needs repair" category.

Validation: 5-Fold Stratified Cross-Validation

5-Fold Stratified Cross-Validation strategy was utilized for validation. This approach was chosen over a simple three-way split to maximize the utility of the available data. Since "needs repair" class is small, Stratified K-Fold ensures that each fold has the same percentage of minority class as the overall data percentage

Logistic Regression Implementation(Baseline Model)

The Logistic Regression model was implemented as the primary baseline to establish a linear performance benchmark. To ensure the mathematical requirements of a linear model were met, Standard Scaling was applied to all numerical features to normalize their range, preventing features with larger magnitudes from dominating the model weights. One-Hot Encoding was utilized to handle categorical variables and string or boolean values. The parameter `handle_unknown='ignore'` was set to ensure the model remains robust when encountering new categories in the test set.

Logistic Regression Hyperparameters

The model was configured with specific hyperparameters to optimize stability and fairness across classes:

- **C=0.01 (Regularization):** A lower C-value was selected for stronger regularization. This applies a higher penalty to the model's coefficients, simplifying the model and reducing the risk of overfitting.
- **class_weight='balanced':** This parameter was used to force the model to pay equal attention to minority classes rather than simply optimizing for the majority class.
- **max_iter=5,000:** The maximum number of iterations was increased to 5,000 to ensure the optimization algorithm reaches convergence, especially given the large number of features created by One-Hot Encoding.

Logistic Regression Evaluation and Conclusion

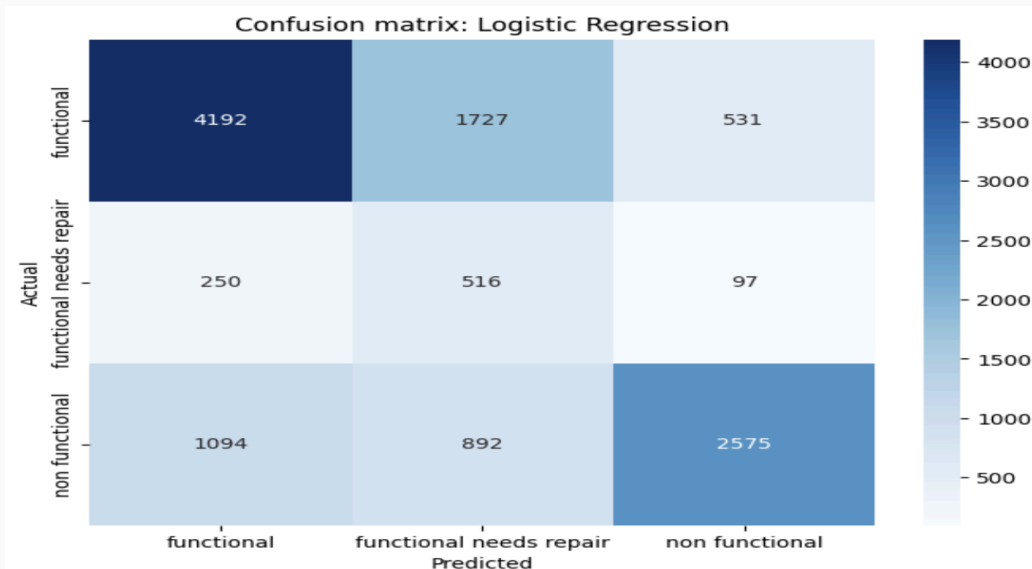
Cross-validation macro-F1: 0.5373 (+/- 0.0033)

=====
Logistic Regression - test set
=====

Accuracy: 0.6134
Macro-F1: 0.5403

Classification report:

	precision	recall	f1-score	support
functional	0.76	0.65	0.70	6450
functional needs repair	0.16	0.60	0.26	863
non functional	0.80	0.56	0.66	4561
accuracy			0.61	11874
macro avg	0.58	0.60	0.54	11874
weighted avg	0.73	0.61	0.65	11874



The model successfully flagged over 60% of the pumps needing repair. If used for planning, this would allow maintenance teams to reach a majority of at-risk sites before they fail completely, potentially saving significant costs in long-term infrastructure replacement.

However, the high number of False Positives (1,727 + 531 functional pumps predicted as needing repair) presents a challenge. Sending a technician to a fully functional pump represents a waste of limited government resources and time.

The confusion matrix shows substantial "leakage" between classes. This suggests that Logistic Regression cannot fully separate the categories, likely due to a data structure that a linear model cannot capture. While Logistic Regression is a strong starting point for identifying general trends, the current error rate is likely too high for effective field-level decision-making.

Random Forest Implementation

A Random Forest Classifier was implemented to capture the complex, non-linear relationships within the data. Unlike linear models, Random Forest is an ensemble method that utilizes multiple decision trees to improve predictive accuracy and control for overfitting. This allows the model to create complex, non-linear boundaries to effectively isolate overlapping groups.

Random Forest Hyperparameters

The model was tuned with specific parameters to balance computational efficiency with predictive depth:

n_estimators=200: An ensemble size of 200 decision trees provides a more stable and robust score for the final classification, reducing the variance typically associated with individual decision trees.

max_depth=20: By limiting the depth of the trees, we introduced a regularization constraint. This prevents the model from perfectly "memorizing" the training data (overfitting) while still allowing enough complexity to identify detailed patterns in geographic and technical features.

class_weight='balanced': This parameter was used to address class imbalance. It ensures the forest penalizes misclassifications of the minority classes more heavily than the majority class.

Notably, while Logistic Regression requires feature scaling, the Random Forest handles different feature scales naturally through its hierarchical splitting mechanism.

Random Forest Evaluation and Conclusion



The Random Forest model achieved a significant 72% reduction in false alarms cases where functional pumps were mislabeled as needing repair—compared to the Logistic Regression baseline. Furthermore, the 27% improvement in the Macro-F1 score confirms that this ensemble model is far more reliable for overall classification.

However, there is a clear trade-off regarding the minority class. The raw count of correctly identified "functional needs repair" cases actually dropped slightly, from 516 to 394. While the overall F1-score improved because the model's predictions are more trustworthy, the recall for this specific minority class decreased.

Additionally, a low standard deviation (± 0.0064) during Cross-Validation indicates that the model is very consistent across different data folds, making it a reliable predictor.

In summary, the Random Forest is highly effective at distinguishing between pumps that require no attention (functional) and those that need immediate action (non-functional). However, the model is currently less effective if the primary goal is to identify the "needs repair" class for

preventative maintenance. Because it misses a larger portion of these cases, it may not yet be the ideal tool for a proactive maintenance strategy aimed at saving money by fixing pumps before they totally fail.

XGBOOST Implementation

While the Random Forest algorithm builds trees in parallel, XGBoost builds them sequentially, where each new tree attempts to correct the errors made by the previous ones. This "boosting" approach is specifically designed to maximize predictive performance on complex, structured datasets.

XGBoost Hyperparameters

The model was configured with parameters focused on balanced learning and preventing overfitting:

`n_estimators=100` & `learning_rate=0.1`: A moderate number of trees combined with a conservative learning rate (shrinkage) ensures that the model learns patterns slowly and accurately. This reduces the risk of overshooting the optimal solution.

`max_depth=6`: By choosing a shallower depth, we prevent individual trees from becoming overly complex, relying instead on the collective correction of the boosting process.

`tree_method="hist"`: The histogram-based algorithm was selected to significantly decrease training time and memory usage. This is essential for a dataset with over 60,000 records and high-cardinality categorical features.

Sample Weighting: To address the class imbalance, `sample_weight="balanced"` was used. The model assigns higher importance to misclassifications within minority groups, ensuring the objective function is not dominated by the majority "functional" class.

XG BOOST Evaluation and Conclusion

Cross-validation macro-F1: 0.6142 (+/- 0.0020)

=====

XGBoost - test set

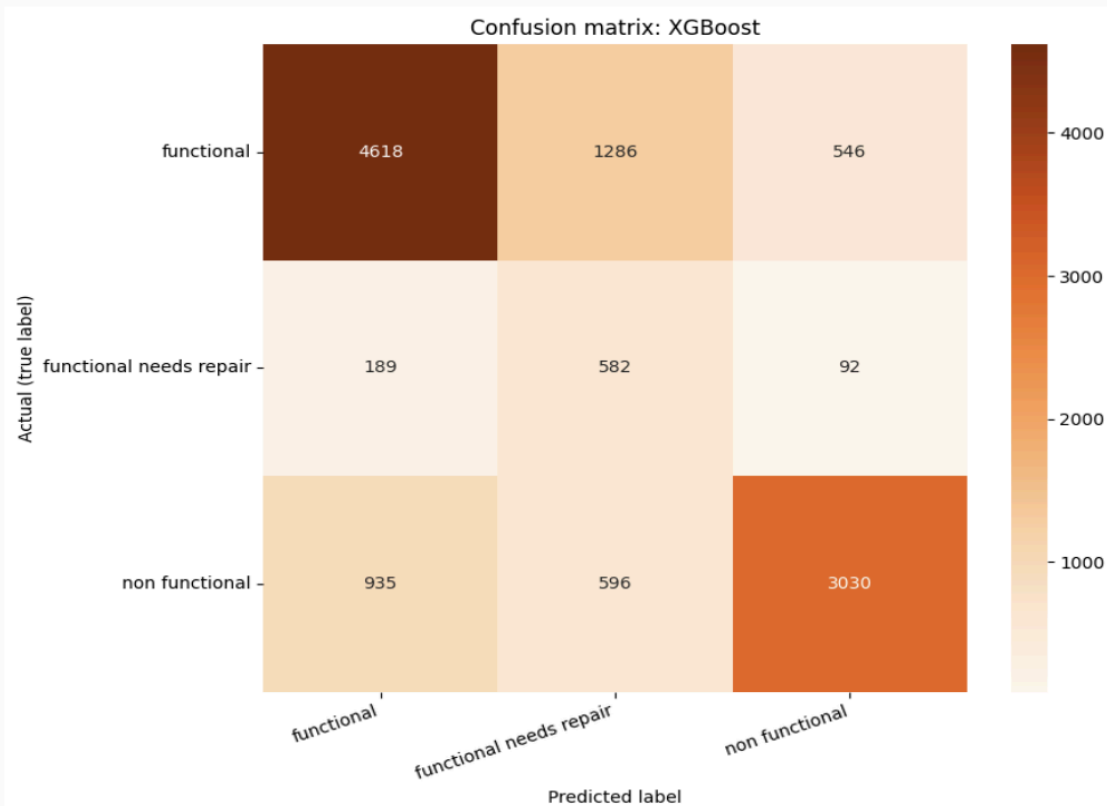
=====

Accuracy: 0.6931
Macro-F1 (equal weight per class): 0.6146
Weighted F1 (by support): 0.7198

Per-class F1 (test):
'functional': 0.7575
'functional needs repair': 0.3499
'non functional': 0.7364

Full classification report:

	precision	recall	f1-score	support
functional	0.80	0.72	0.76	6450
functional needs repair	0.24	0.67	0.35	863
non functional	0.83	0.66	0.74	4561
accuracy			0.69	11874
macro avg	0.62	0.68	0.61	11874
weighted avg	0.77	0.69	0.72	11874



While the XGBoost Macro-F1 (0.6146) is lower than the Random Forest (0.68), it succeeded where other models failed: it is much more aggressive at catching the "needs repair" class. This demonstrates that the gradient boosting process, combined with sample weighting, effectively "forced" the model to learn the patterns of this difficult minority class.

However, there is a spike in "False Alarms," with 1,286 functional pumps being predicted as "needs repair." This explains the drop in the overall Macro-F1; the model is making more predictions in this category to ensure it does not miss failing pumps. Despite this, a low standard deviation (± 0.0020) indicates that XGBoost is very consistent across different data folds during Cross-Validation, making it a reliable predictor.

Because this model identifies nearly 68% of the pumps needing repair (a Recall of 0.67 for that specific class), it is the best tool for preventing total pump failure. By catching 582 cases early, the ministry can perform low-cost maintenance instead of paying for high-cost reconstruction after a total collapse.

Operationally, this model requires a higher budget for field inspections due to the 1,286 false positives. However, the business argument is clear: the cost of a "wasted" inspection is significantly lower than the cost of a community losing water access for weeks while a replacement pump is sourced.

Binary Classification Approach

Based on the performance of both the Random Forest and XGBoost models for 3-classes, it is evident that while each model possesses distinct strengths, neither serves as a perfect "all-in-one" solution for the diverse challenges within the Taarifa waterpoint dataset.

In the binary class framework, the "Non-Functional" and "Functional Needs Repair" categories are consolidated into a single "Action Required" class. Combining these two categories addresses the primary bottleneck of class imbalance encountered in previous models.

This binary approach allows the model to focus on a much clearer signal: whether the pump is healthy or not. From the perspective of a field technician, the operational outcome remains the same, the model simply determines if a maintenance team should be dispatched to the site or not. This model focuses on Site Prioritization rather than mechanical diagnosis.

Binary Classification Evaluation and Conclusion

I have implemented 2 models XGBoost and Random forest for 2-class classification

```

=== Random Forest (binary) – held-out test (tuned threshold) ===
Accuracy: 0.8168
Macro precision: 0.8182 | Macro recall: 0.8121 | Macro-F1: 0.8139

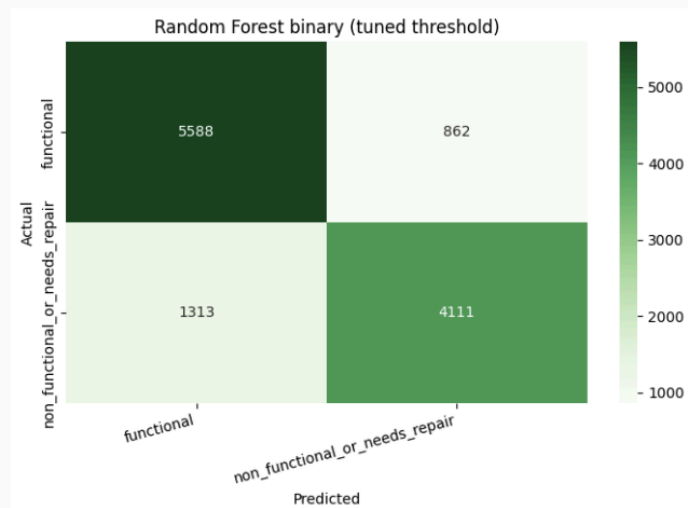
```

Classification report:

	precision	recall	f1-score	support
functional	0.81	0.87	0.84	6450
non_functional_or_needs_repair	0.83	0.76	0.79	5424
accuracy			0.82	11874
macro avg	0.82	0.81	0.81	11874
weighted avg	0.82	0.82	0.82	11874

Confusion matrix (rows=actual, cols=pred):

	pred:functional \	
actual:functional	5588	
actual:non_functional_or_needs_repair	1313	
	pred:non_functional_or_needs_repair	
actual:functional		862
actual:non_functional_or_needs_repair		4111



```

=== XGBoost (binary) – held-out test (tuned threshold) ===
Accuracy: 0.8114
Macro precision: 0.8142 | Macro recall: 0.8057 | Macro-F1: 0.8079

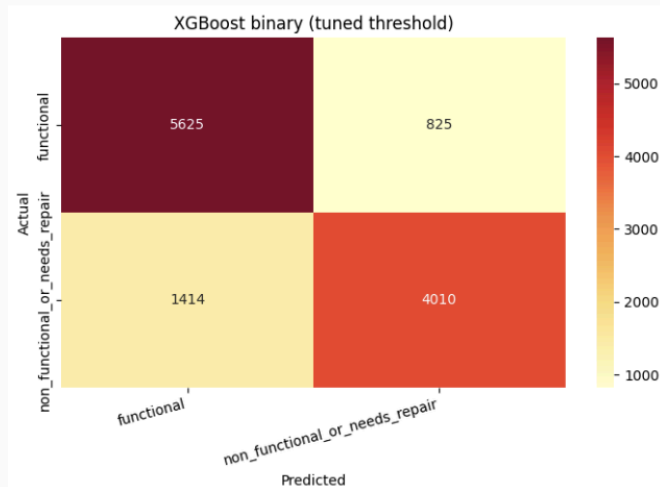
```

Classification report:

	precision	recall	f1-score	support
functional	0.80	0.87	0.83	6450
non_functional_or_needs_repair	0.83	0.74	0.78	5424
accuracy			0.81	11874
macro avg	0.81	0.81	0.81	11874
weighted avg	0.81	0.81	0.81	11874

Confusion matrix (rows=actual, cols=pred):

	pred:functional \	
actual:functional	5625	
actual:non_functional_or_needs_repair	1414	
	pred:non_functional_or_needs_repair	
actual:functional		825
actual:non_functional_or_needs_repair		4010



Both models showed a significant leap in performance upon transitioning to binary classification. Random Forest achieved an accuracy of 81.68%, while XGBoost reached 81.14%. Ultimately, the Random Forest model is preferred for this analysis because it offers a higher recall (0.76) for the "Action Required" class compared to XGBoost (0.74). In this context, maximizing the identification of non-functional or failing pumps is the priority.

With a precision of 83%, the model ensures that for every 10 teams sent to the field, 8 will find a pump that truly requires intervention. This drastically minimizes resource wastage compared to previous multi-class models, which were prone to high false-alarm rates.

From an operational standpoint, the most critical decision is whether or not to dispatch a technician to a site. Once on-site, a technician can accurately diagnose whether the issue requires a minor repair or a full replacement. By consolidating these categories, the model focuses the budget on getting the right people to the right places.

The model successfully captures over 76% of all pumps needing attention. This ensures that the vast majority of communities who don't have access to water access are visible to the Ministry, allowing for proactive infrastructure planning rather than reactive crisis response.

Executive Summary

This Executive Summary is crafted to provide insights from both a statistical and business point of view so that both technical and non-technical individuals can understand and leverage the information.

Insights from data:

About Data: There is severe **class imbalance**, with the "functional needs repair" category comprising only 7.3% of observations. Around 35% of data points are missing in some important fields like population and construction year, which play an important role in identifying the health of a waterpoint. From this, we can conclude that the survey team needs to be trained properly for data collection. This will help in the future to build better models and improve prediction.

About Model Implementation:

For 3-class classification: While the Random Forest model achieved the highest overall Macro-F1 score (0.68) compared to the Logistic Regression baseline, it struggled with the minority class. The XGBoost model proved more aggressive, catching the "needs repair" cases, making it superior for preventative action. However, choosing between Random Forest and XGBoost depends on the primary goal of the management. If the goal is to capture as many "needs repair" cases as possible, then XGBoost is better, but if the team's goal is to capture non-functional/needs repair waterpoints while avoiding too many false alarms, then the 2-class classification approach is superior.

Changing the approach to binary classification: Consolidating the three categories into a binary "Action Required" class (non-functional or needs repair) yielded the most practical performance leap. The final **Random Forest** binary model achieved 81.68% accuracy with a precision of **83%** and a recall of **76%** for the "Action Required" class.

Business-Driven Use Cases and Recommendations:

The choice of model depends on management's strategic priority:

For Proactive Maintenance of "Needs Repair" Class: If the management's priority is to capture the "needs repair" class with urgency, as capturing these cases on time can save the significant cost of replacing the waterpoint altogether, **the XGBoost (3-Class)** model is recommended. Its aggressive prediction for the "needs repair" class allows the Ministry to flag failing pumps for low-cost maintenance before they break entirely. However, the drawback is the high number of false positives (functional pumps marked as "needs repair"). If the cost of sending a maintenance team is not significant, and repairing the waterpoints can save substantial money, then this is a perfect choice for management decision-making.

For Operational Efficiency & Immediate Crisis Response: If management's goal is to provide clean water to communities, and there is no significant difference in the cost of replacing or repairing the waterpoint, and the cost of sending a maintenance team is high, then the

superior choice is to proceed with 2-class classification. This model ensures that waterpoints needing attention ("needs repair" and non-functional) are captured, but not at the cost of too many false positives. This ensures high confidence that 8 out of every 10 dispatched teams will find a pump requiring intervention.

References/Bibliography

Taarifa. Taarifa Waterpoints Dashboard. <https://taarifa.org>.

Taarifa. Taarifa GitHub Organization. <https://github.com/taarifa>.